R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Gestion des processus, cycle de vie et communication inter-processus via les signaux POSIX sous Unix/Linux

Synthèse exécutive : Ce cours pose les fondements de la gestion des tâches au sein des systèmes d'exploitation de type Unix/Linux. Il explique la transition critique entre un programme passif stocké sur disque dur et un processus actif résidant en mémoire RAM. Pour faire fonctionner de nombreuses applications simultanément sur une même machine, le noyau (kernel) s'appuie sur le multitâche préemptif, gérant le cycle de vie des processus à travers trois états fondamentaux (Élu, Prêt, Bloqué). Le cours détaille la filiation arborescente des processus (PID, PPID) issue du processus racine systemd/init (PID 1), ainsi que l'utilisation du mécanisme universel d'interruption asynchrone : les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie et Attributs System

Pour qu'un ordinateur exécute du code, le système d'exploitation doit faire basculer le programme (données passives sur disque) vers l'état de processus (entité dynamique en RAM). L'ordonnanceur du noyau gère cette exécution à travers un cycle de vie strict composé de trois états principaux : 1. **Élu (Running)** : Le processus occupe actuellement le processeur (CPU) et ses instructions sont exécutées. 2. **Prêt (Ready)** : Le processus possède toutes ses ressources en mémoire RAM mais attend que l'ordonnanceur lui attribue une tranche de temps CPU. 3. **Bloqué (Waiting / Blocked)** : Le processus est suspendu car il attend un événement externe (ex: lecture de fichier sur disque, frappe clavier de l'utilisateur, paquet réseau). Dès que l'événement survient, il ne repasse pas directement 'Élu', mais retourne dans l'état 'Prêt'. Tous les processus sous Linux dérivent d'une structure parent-enfant. Lors du démarrage, le noyau crée le tout premier processus utilisateur, `systemd` (ou `init`), ayant le PID 1. Lorsque vous lancez une commande dans un terminal, le terminal (parent) effectue un `fork()` pour se cloner, puis l'enfant exécute `exec()` pour charger le nouveau programme.

- Programme vs Processus : Un programme est un fichier exécutable passif stocké sur disque. Un processus est son instance dynamique en mémoire RAM avec du temps CPU et des ressources allouées.
- Multitâche Préemptif : Mécanisme où le noyau attribue et retire dynamiquement des tranches de temps CPU (quanta) aux processus pour simuler la simultanéité.
- PCB (Process Control Block) : Structure de données interne du noyau contenant la carte mémoire, la priorité, l'état, l'identifiant de l'utilisateur (UID) et les descripteurs de fichiers de chaque processus.
- Arborescence PID/PPID : Chaque processus possède un identifiant unique (PID - Process ID) et pointe vers le PID de son créateur (PPID - Parent Process ID). Le processus racine est systemd/init (PID 1).
- Appels système fork() et exec() : fork() duplique le processus parent à l'identique, puis exec() remplace le code du processus enfant par un nouveau programme exécutable.

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme est un livre de recettes sagement rangé dans une bibliothèque (passif sur le disque dur, ne consomme ni électricité ni ingrédients). Le processus est le cuisinier en train de préparer activement la recette dans sa cuisine (dynamique en RAM, consomme de l'énergie, de l'espace sur le plan de travail et utilise des ustensiles).*

2. Contrôle, Communication Inter-Processus et Signaux POSIX

Les signaux POSIX constituent le moyen le plus simple et le plus ancien de communication entre processus (IPC). Lorsqu'un signal parvient à un processus cible, son exécution est temporairement interrompue pour qu'il traite l'événement via un 'gestionnaire de signal' (signal handler). Un développeur peut programmer son application pour intercepter (catch) un signal ou l'ignorer (ignore). Par exemple, un éditeur de texte peut intercepter SIGINT ou

SIGTERM pour afficher un dialogue 'Voulez-vous sauvegarder votre travail ?' avant de s'arrêter. Cependant, les spécifications POSIX imposent une règle absolue : les signaux **SIGKILL (9)** et **SIGSTOP (19)** ne peuvent JAMAIS être interceptés, masqués ou ignorés. Le noyau Linux traite directement ces deux signaux, garantissant ainsi à l'administrateur système de pouvoir arrêter un processus malveillant ou bloqué dans une boucle infinie.

- Signal POSIX : Interruption logicielle asynchrone envoyée à un processus pour l'avertir d'un événement ou exiger une action.
- SIGINT (Signal 2) : Envoyé par la combinaison 'Ctrl + C'. Demande l'interruption douce d'un processus interactif.
- SIGTERM (Signal 15) : Signal d'arrêt par défaut de la commande 'kill'. Demande une fermeture propre en autorisant le nettoyage des ressources.
- SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel traité directement par le noyau. Il détruit immédiatement le processus sans sommation.
- Job Control (SIGSTOP / SIGCONT) : 'Ctrl + Z' envoie SIGSTOP pour suspendre l'exécution. Les commandes 'fg' (foreground) ou 'bg' (background) envoient SIGCONT pour reprendre l'exécution.

Cas concret enseignant : *L'analogie de l'évacuation de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander d'évacuer (vous avez le temps de ranger votre bureau, sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (9) est une intervention de sécurité qui enfonce la porte et vous expulsionne immédiatement sur-le-champ sans vous laisser la possibilité de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : Sans le tiret '-', le programme 'kill' interprète '9' non pas comme le numéro du signal SIGKILL, mais comme un premier identifiant de processus (PID 9). La commande tente donc d'envoyer le signal par défaut (SIGTERM 15) au processus PID 9 (souvent inexistant ou protégé) ET au processus PID 1234. Le tiret '-9' indique explicitement l'option décrivant le signal 9 (SIGKILL) à envoyer au PID 1234.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie est un processus qui a terminé son exécution mais dont le père n'a pas encore lu le code de retour (via l'appel système wait()). Il ne consomme ni RAM ni CPU, mais conserve une entrée dans la table des processus du noyau. On NE PEUT PAS tuer un Zombie avec 'kill -9' car il est DÉJÀ mort ! Pour faire disparaître un Zombie, il faut forcer le processus père à lire son état (en lui envoyant le signal SIGCHLD), ou bien tuer le processus père pour que 'systemd/init' adopte le Zombie et le nettoie immédiatement (opération de 'reaping').

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, un utilisateur exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez ce qui se passe lors de l'envoi du signal -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' indique au shell de lancer la tâche en arrière-plan (background). Le shell affiche l'identifiant de travail (job ID) ainsi que le PID attribué (ex: [1] 4521) et rend immédiatement la main à l'utilisateur pour qu'il puisse saisir d'autres commandes. Étape 2 : La commande 'ps -ef | grep sleep' parcourt l'intégralité de la table des processus du système et en extrait la ligne correspondant à l'exécution de la commande 'sleep'. Cela permet d'identifier visuellement son PID exact (4521) ainsi que son PPID (qui correspond au PID du shell Bash en cours). Étape 3 : La commande 'kill -15 4521' envoie le signal SIGTERM (Signal 15) au processus 4521. Comme 'sleep' ne possède pas de gestionnaire spécifique pour capturer ou ignorer SIGTERM, il applique l'action par défaut associée à ce signal : l'arrêt propre et immédiat du programme.

Résultat : Le processus 'sleep 600' est détruit proprement. Une nouvelle exécution de la commande 'ps -ef | grep sleep' confirme sa disparition définitive de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du premier processus utilisateur démarré par le noyau Linux ?

R : Il s'agit du processus 'systemd' (ou 'init' sur les anciens systèmes), et son PID est impérativement 1.

Q : Quel signal POSIX est généré et transmis au processus au premier plan lorsqu'un utilisateur tape la combinaison 'Ctrl + C' dans un terminal ?

R : La combinaison 'Ctrl + C' génère le signal SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : Parce que SIGKILL n'est pas transmis à l'espace utilisateur du processus target : il est directement capturé et exécuté par le noyau lui-même, garantissant un arrêt forcé de l'application.